

Методичка 2. JSX и компоненты React

Цель занятия

На прошлом занятии мы разобрали архитектуру React Vite проекта и поняли главную цепочку:

text

```
index.html -> src/main.jsx -> src/App.jsx -> интерфейс в браузере
```

Теперь переходим внутрь файла `src/App.jsx`. Главная цель занятия - понять, что такое JSX, что такое компонент React и почему интерфейс в React собирается из небольших функций.

После занятия должны уметь:

- объяснить, что компонент React - это JavaScript-функция, которая возвращает JSX;
- отличать HTML-теги от React-компонентов;
- понимать, почему имена компонентов пишутся с большой буквы;
- использовать `className` вместо `class`;
- вставлять JavaScript-выражения в JSX через фигурные скобки `{}`;
- выносить часть интерфейса в отдельный компонент;
- импортировать и экспортировать компоненты.

1. Что говорит документация React

В актуальной документации React компоненты описываются как строительные блоки пользовательского интерфейса. React позволяет объединять разметку, стили и JavaScript-логику в собственные переиспользуемые элементы интерфейса.

Очень коротко:

text

```
Компонент React = JavaScript-функция + JSX-разметка
```

Пример простого компонента:

javascript

```
function Welcome() {  
  return <h1>Привет, React!</h1>  
}
```

Компонент можно использовать почти как HTML-тег:

javascript

```
<Welcome />
```

Но это не обычный HTML-тег. Это вызов нашего собственного React-компонента.

2. Что такое компонент

Компонент - это самостоятельная часть интерфейса.

Например, в приложении могут быть такие компоненты:

```
text
  Header
  Button
  Card
  Profile
  Menu
  Footer
```

Если представить сайт как конструктор, то компонент - это одна деталь конструктора. Из маленьких деталей собирается большая страница.

В нашем проекте уже есть главный компонент:

```
javascript
function App() {
  const [count, setCount] = useState(0)

  return (
    <>
      {/* интерфейс приложения */}
    </>
  )
}
```

App - это компонент. Он является обычной JavaScript-функцией, но возвращает JSX.

3. Почему компонент начинается с большой буквы

React различает обычные HTML-теги и пользовательские компоненты по первой букве.

Обычные HTML-теги пишутся с маленькой буквы:

```
javascript
<section>
<div>
<h1>
<p>
<button>
<img>
```

React-компоненты пишутся с большой буквы:

```
javascript
<App />
<Hero />
<CounterButton />
<DocsLinks />
```

Если написать компонент с маленькой буквы, React будет считать его обычным HTML-тегом.

Неправильно:

```
javascript
function hero() {
  return <h1>Hero</h1>
}
```

Правильно:

```
javascript
function Hero() {
  return <h1>Hero</h1>
}
```

Правило:

```
text
Имена React-компонентов всегда начинаются с большой буквы.
```

4. Что такое JSX

JSX - это синтаксис, который позволяет писать разметку внутри JavaScript-файла.

Пример из обычного HTML:

```
html
<button class="counter">
  Count is 0
</button>
```

Похожий пример в JSX:

```
javascript
<button className="counter">
  Count is {count}
</button>
```

JSX похож на HTML, но это не HTML. JSX превращается в JavaScript-код, который React использует для создания интерфейса.

В документации React подчеркивается: компонент может содержать разметку, которую React затем отображает в браузере.

5. Главное отличие JSX от HTML

Отличие 1. `className` вместо `class`

В HTML мы пишем:

```
html
  <button class="counter">Нажми</button>
```

В JSX нужно писать:

```
javascript
  <button className="counter">Нажми</button>
```

Почему так? JSX ближе к JavaScript, а `class` в JavaScript - служебное слово для создания классов. Поэтому React использует имя `className`.

В нашем проекте:

```
javascript
  <button
    type="button"
    className="counter"
    onClick={() => setCount((count) => count + 1)}
  >
    Count is {count}
  </button>
```

Класс `counter` связывает кнопку со стилями из `App.css`:

```
css
  .counter {
    font-size: 16px;
    padding: 5px 10px;
  }
```

Отличие 2. JavaScript вставляется через `{}`

В JSX можно вставлять значения переменных, результаты вычислений и вызовы функций.

Пример:

```
javascript
  const userName = 'Анна'

  function Greeting() {
    return <h1>Привет, {userName}!</h1>
  }
```

На странице появится:

```
text
  Привет, Анна!
```

В нашем проекте:

```
javascript
  Count is {count}
```

`count` - это JavaScript-переменная из `useState`. Фигурные скобки говорят React: возьми значение переменной и вставь его в интерфейс.

Отличие 3. Самозакрывающиеся теги

В HTML иногда можно написать так:

```
html
  
```

В JSX тег без содержимого нужно закрывать:

```
javascript
  <img src={heroImg} alt="" />
```

Так же закрываются пользовательские компоненты без вложенного содержимого:

```
javascript
  <Hero />
```

Отличие 4. Нужно возвращать один общий корень

Компонент не может вернуть два соседних элемента без общей обертки.

Неправильно:

```
javascript
function Example() {
  return (
    <h1>Заголовок</h1>
    <p>Текст</p>
  )
}
```

Правильно через div:

```
javascript
function Example() {
  return (
    <div>
      <h1>Заголовок</h1>
      <p>Текст</p>
    </div>
  )
}
```

Правильно через фрагмент:

```
javascript
function Example() {
  return (
    <>
      <h1>Заголовок</h1>
      <p>Текст</p>
    </>
  )
}
```

Фрагмент `<>...</>` не создает лишний `div` в HTML. Он просто помогает сгруппировать несколько элементов.

В нашем `App.jsx` используется именно фрагмент:

```
javascript
return (
  <>
    <section id="center">
      {/* содержимое */}
    </section>

    <div className="ticks"></div>

    <section id="next-steps">
      {/* содержимое */}
    </section>
  </>
)
```

6. JSX внутри `return`

Компонент обычно возвращает JSX через `return`.

Если JSX занимает одну строку, можно написать так:

```
javascript
function Title() {
  return <h1>React</h1>
}
```

Если JSX занимает несколько строк, его нужно обернуть в круглые скобки:

```
javascript
function Title() {
  return (
    <h1>
      React
    </h1>
  )
}
```

Это важно из-за правил JavaScript. Если написать перенос строки сразу после `return`, код может работать не так, как ожидается.

Неправильно:

```
javascript
function Title() {
  return
    <h1>React</h1>
}
```

Правильно:

```
javascript
function Title() {
  return (
    <h1>React</h1>
  )
}
```

7. Разбор JSX из текущего проекта

Посмотрим на небольшой фрагмент из `src/App.jsx`:

```
javascript
<section id="center">
  <div className="hero">
    <img src={heroImg} className="base" width="170" height="179" alt="" />
    <img src={reactLogo} className="framework" alt="React logo" />
    <img src={viteLogo} className="vite" alt="Vite logo" />
  </div>
  <div>
    <h1>Get started</h1>
    <p>
      Edit <code>src/App.jsx</code> and save to test <code>HMR</code>
    </p>
  </div>
</section>
```

Что здесь есть:

- `<section>` - обычный HTML-тег;
- `id="center"` - обычный атрибут;
- `<div className="hero">` - блок со стилями из CSS;
- `<img />` - изображения;
- `src={heroImg}` - значение берется из JavaScript-переменной;
- `<h1>` - заголовок;
- `<p>` - абзац;
- `<code>` - тег для отображения фрагмента кода.

Файл `App.jsx` выше импортирует изображения:

```
javascript
import reactLogo from './assets/react.svg'
import viteLogo from './assets/vite.svg'
import heroImg from './assets/hero.png'
```

Поэтому в JSX можно писать:

```
javascript
<img src={heroImg} alt="" />
```

Фигурные скобки здесь обязательны, потому что `heroImg` - это переменная.

8. Разделяем интерфейс на компоненты

Сейчас `App.jsx` содержит сразу много JSX. Для маленького шаблона это допустимо, но по мере роста проекта файл может стать слишком длинным.

React предлагает разбивать интерфейс на компоненты.

Например, hero-блок можно вынести в отдельный компонент:

javascript

```
function Hero() {
  return (
    <div className="hero">
      <img src={heroImg} className="base" width="170" height="179" alt="" />
      <img src={reactLogo} className="framework" alt="React logo" />
      <img src={viteLogo} className="vite" alt="Vite logo" />
    </div>
  )
}
```

После этого внутри `App` можно использовать:

javascript

```
<Hero />
```

Было:

javascript

```
<section id="center">
  <div className="hero">
    <img src={heroImg} className="base" width="170" height="179" alt="" />
    <img src={reactLogo} className="framework" alt="React logo" />
    <img src={viteLogo} className="vite" alt="Vite logo" />
  </div>
  <div>
    <h1>Get started</h1>
    <p>
      Edit <code>src/App.jsx</code> and save to test <code>HMR</code>
    </p>
  </div>
</section>
```

Стало:

javascript

```
<section id="center">
  <Hero />
  <div>
    <h1>Get started</h1>
    <p>
      Edit <code>src/App.jsx</code> and save to test <code>HMR</code>
    </p>
  </div>
</section>
```

Код стал короче и понятнее: по названию `Hero` видно, что здесь находится hero-блок.

9. Компонент внутри того же файла

На первых этапах можно создавать компоненты прямо в `App.jsx`.

Пример:

```
javascript
function Hero() {
  return (
    <div className="hero">
      <img src={heroImg} className="base" width="170" height="179" alt="" />
      <img src={reactLogo} className="framework" alt="React logo" />
      <img src={viteLogo} className="vite" alt="Vite logo" />
    </div>
  )
}

function App() {
  const [count, setCount] = useState(0)

  return (
    <>
      <section id="center">
        <Hero />
        <h1>Get started</h1>
      </section>
    </>
  )
}
```

Здесь в одном файле два компонента:

- `Hero` отвечает за изображения в верхнем блоке;
- `App` отвечает за всю страницу.

Важно: компонент `Hero` находится на верхнем уровне файла, а не внутри функции `App`.

Так лучше:

```
javascript
function Hero() {
  return <div className="hero">...</div>
}

function App() {
  return <Hero />
}
```

Так лучше не делать:

```
javascript
function App() {
  function Hero() {
    return <div className="hero">...</div>
  }

  return <Hero />
}
```

Для учебных задач второй вариант может работать, но он хуже читается и мешает переиспользованию компонента.

10. Компонент в отдельном файле

Когда компонентов становится больше, их удобно переносить в отдельные файлы.

Например, можно создать файл:

```
text
src/Hero.jsx
```

Внутри:

```
javascript
import reactLogo from './assets/react.svg'
import viteLogo from './assets/vite.svg'
import heroImg from './assets/hero.png'

function Hero() {
  return (
    <div className="hero">
      <img src={heroImg} className="base" width="170" height="179" alt="" />
      <img src={reactLogo} className="framework" alt="React logo" />
      <img src={viteLogo} className="vite" alt="Vite logo" />
    </div>
  )
}

export default Hero
```

А в App.jsx импортировать:

```
javascript
import Hero from './Hero.jsx'
```

И использовать:

```
javascript
<Hero />
```

Связь файлов:

```
text
src/App.jsx
└─ импортирует src/Hero.jsx
    └─ импортирует изображения из src/assets/
```

11. Export и import простыми словами

`export` делает компонент доступным для других файлов.

```
javascript
export default Hero
```

`import` подключает компонент в другом файле.

```
javascript
import Hero from './Hero.jsx'
```

Если провести аналогию:

```
text
export = "разрешаю использовать наружу"
import = "подключаю сюда"
```

В текущем проекте `App.jsx` уже экспортирует компонент:

```
javascript
export default App
```

А `main.jsx` его импортирует:

```
javascript
import App from './App.jsx'
```

Потом `main.jsx` отображает компонент:

```
javascript
<App />
```

12. Вложенность компонентов

Компоненты можно вкладывать друг в друга.

Пример:

```
javascript
function Page() {
  return (
    <main>
      <Header />
      <Content />
      <Footer />
    </main>
  )
}
```

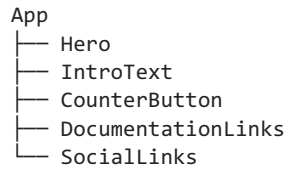
Это похоже на HTML-вложенность:

```
html
<main>
  <header></header>
  <section></section>
  <footer></footer>
</main>
```

Но в React мы можем создавать собственные теги-компоненты: `Header`, `Content`, `Footer`.

В нашем проекте можно мысленно разделить App так:

text



Такой подход помогает держать код в порядке.

13. Что можно писать внутри JSX

В JSX можно писать:

Текст

javascript

```
<h1>Get started</h1>
```

HTML-подобные теги

javascript

```
<section>
  <p>Текст страницы</p>
</section>
```

Свои компоненты

javascript

```
<Hero />
```

JavaScript-выражения в `{}`

javascript

```
<p>Счетчик: {count}</p>
```

Значения атрибутов из переменных

javascript

```
<img src={heroImg} alt="" />
```

Обработчики событий

javascript

```
<button onClick={() => setCount(count + 1)}>
  Нажать
</button>
```

Важно: внутри `{}` можно писать выражения, которые возвращают значение. Например:

javascript

```
{count + 1}
{userName}
{isActive ? 'Активно' : 'Неактивно'}
```

Но нельзя просто написать обычную инструкцию `if` прямо внутри JSX так:

javascript

```
<p>{if (count > 0) 'Есть клики'}</p>
```

Условный рендеринг мы отдельно разберем в следующих методичках.

14. Комментарии в JSX

Обычный JavaScript-комментарий:

```
javascript
// Это комментарий
```

Внутри JSX комментарий пишется так:

```
javascript
function App() {
  return (
    <>
      {/* Это комментарий внутри JSX */}
      <h1>React</h1>
    </>
  )
}
```

В реальном коде не стоит комментировать каждую очевидную строку.

15. Практический пример: выносим кнопку в компонент

В App.jsx сейчас есть кнопка:

```
javascript
<button
  type="button"
  className="counter"
  onClick={() => setCount((count) => count + 1)}
>
  Count is {count}
</button>
```

Можно вынести ее в отдельный компонент CounterButton.

Пока сделаем простой вариант в том же файле (вне App):

```
javascript
function CounterButton({ count, onClick }) {
  return (
    <button type="button" className="counter" onClick={onClick}>
      Count is {count}
    </button>
  )
}
```

А внутри App использовать:

```
javascript
<CounterButton
  count={count}
  onClick={() => setCount((count) => count + 1)}
/>
```

Здесь появляются props - данные, которые родительский компонент передает дочернему.

Сейчас достаточно понять:

```
text
App хранит count
App передает count в CounterButton
CounterButton показывает count на кнопке
```

Подробно props мы разберем отдельно.

16. Практический пример: выносим hero-блок

Ниже пример полного фрагмента `App.jsx` после выноса hero-блока:

```
javascript
import { useState } from 'react'
import reactLogo from './assets/react.svg'
import viteLogo from './assets/vite.svg'
import heroImg from './assets/hero.png'
import './App.css'

function Hero() {
  return (
    <div className="hero">
      <img src={heroImg} className="base" width="170" height="179" alt="" />
      <img src={reactLogo} className="framework" alt="React logo" />
      <img src={viteLogo} className="vite" alt="Vite logo" />
    </div>
  )
}

function App() {
  const [count, setCount] = useState(0)

  return (
    <>
      <section id="center">
        <Hero />
        <div>
          <h1>Get started</h1>
          <p>
            Edit <code>src/App.jsx</code> and save to test <code>HMR</code>
          </p>
        </div>
        <button
          type="button"
          className="counter"
          onClick={() => setCount((count) => count + 1)}
        >
          Count is {count}
        </button>
      </section>
    </>
  )
}

export default App
```

Здесь `Hero` - отдельный компонент, но он пока находится в том же файле.

17. Частые ошибки

Ошибка 1. Компонент с маленькой буквы

Неправильно:

```
javascript
function hero() {
  return <h1>Hero</h1>
}
```

Правильно:

```
javascript
function Hero() {
  return <h1>Hero</h1>
}
```

Ошибка 2. Использование `class` вместо `className`

Неправильно:

```
javascript
<div class="hero"></div>
```

Правильно:

```
javascript
<div className="hero"></div>
```

Ошибка 3. Забыли закрыть тег

Неправильно:

```
javascript
<img src={heroImg} alt="">
```

Правильно:

```
javascript
<img src={heroImg} alt="" />
```

Ошибка 4. Несколько корневых элементов без обертки

Неправильно:

```
javascript
function App() {
  return (
    <h1>React</h1>
    <p>Учебный проект</p>
  )
}
```

Правильно:

```
javascript
function App() {
  return (
    <>
      <h1>React</h1>
      <p>Учебный проект</p>
    </>
  )
}
```

Ошибка 5. Перенос строки после `return`

Неправильно:

```
javascript
function App() {
  return
    <h1>React</h1>
}
```

Правильно:

```
javascript
function App() {
  return (
    <h1>React</h1>
  )
}
```

Ошибка 6. Строка вместо переменной

Неправильно:

```
javascript

```

Правильно:

```
javascript
<img src={heroImg} alt="" />
```

В первом случае React передаст строку "heroImg". Во втором случае React возьмет значение переменной heroImg.

18. Практика

Задание 1

В файле `src/App.jsx` создайте компонент `PageTitle`:

```
javascript
function PageTitle() {
  return (
    <div>
      <h1>Get started</h1>
      <p>
        Edit <code>src/App.jsx</code> and save to test <code>HMR</code>
      </p>
    </div>
  )
}
```

Замените старый блок:

```
javascript
<div>
  <h1>Get started</h1>
  <p>
    Edit <code>src/App.jsx</code> and save to test <code>HMR</code>
  </p>
</div>
```

на:

```
javascript
<PageTitle />
```

Задание 2

Создайте компонент `Hero` и перенесите в него блок с изображениями:

```
javascript
function Hero() {
  return (
    <div className="hero">
      <img src={heroImg} className="base" width="170" height="179" alt="" />
      <img src={reactLogo} className="framework" alt="React logo" />
      <img src={viteLogo} className="vite" alt="Vite logo" />
    </div>
  )
}
```

Внутри `App` замените старый `hero`-блок на:

```
javascript
<Hero />
```

Задание 3

Создайте компонент CounterButton:

```
javascript
function CounterButton({ count, onClick }) {
  return (
    <button type="button" className="counter" onClick={onClick}>
      Count is {count}
    </button>
  )
}
```

Внутри App замените старую кнопку на:

```
javascript
<CounterButton
  count={count}
  onClick={() => setCount((count) => count + 1)}
/>
```

Задание 4

Запустите проект:

```
bash
npm run dev
```

Проверьте, что страница выглядит так же, как до разбиения на компоненты.

Задание 5

Ответьте: стало ли легче читать App.jsx после выноса частей интерфейса в компоненты? Почему?

19. Контрольные вопросы

1. Что такое React-компонент?
2. Почему компонент должен начинаться с большой буквы?
3. Что такое JSX?
4. Почему в JSX пишется className, а не class?
5. Для чего нужны фигурные скобки {} в JSX?
6. Почему тег в JSX нужно закрывать?
7. Что такое фрагмент <>...</>?
8. Почему JSX после return обычно оборачивают в круглые скобки?
9. Чем отличается <section> от <Hero />?
10. Что делает export default?
11. Что делает import?
12. Зачем разбивать интерфейс на компоненты?
13. Где можно создать компонент на первых занятиях?
14. Когда компонент лучше вынести в отдельный файл?
15. Что такое props в самом простом объяснении?

20. Краткий итог

JSX позволяет писать HTML-подобную разметку внутри JavaScript. React-компонент - это JavaScript-функция, которая возвращает JSX. Компоненты помогают разбивать интерфейс на понятные части и переиспользовать их.

Главные правила занятия:

text

Компонент пишется с большой буквы.

Компонент возвращает JSX.

В JSX используется `className`.

JavaScript-значения вставляются через `{}`.

Несколько элементов нужно оборачивать в общий корень или фрагмент.